

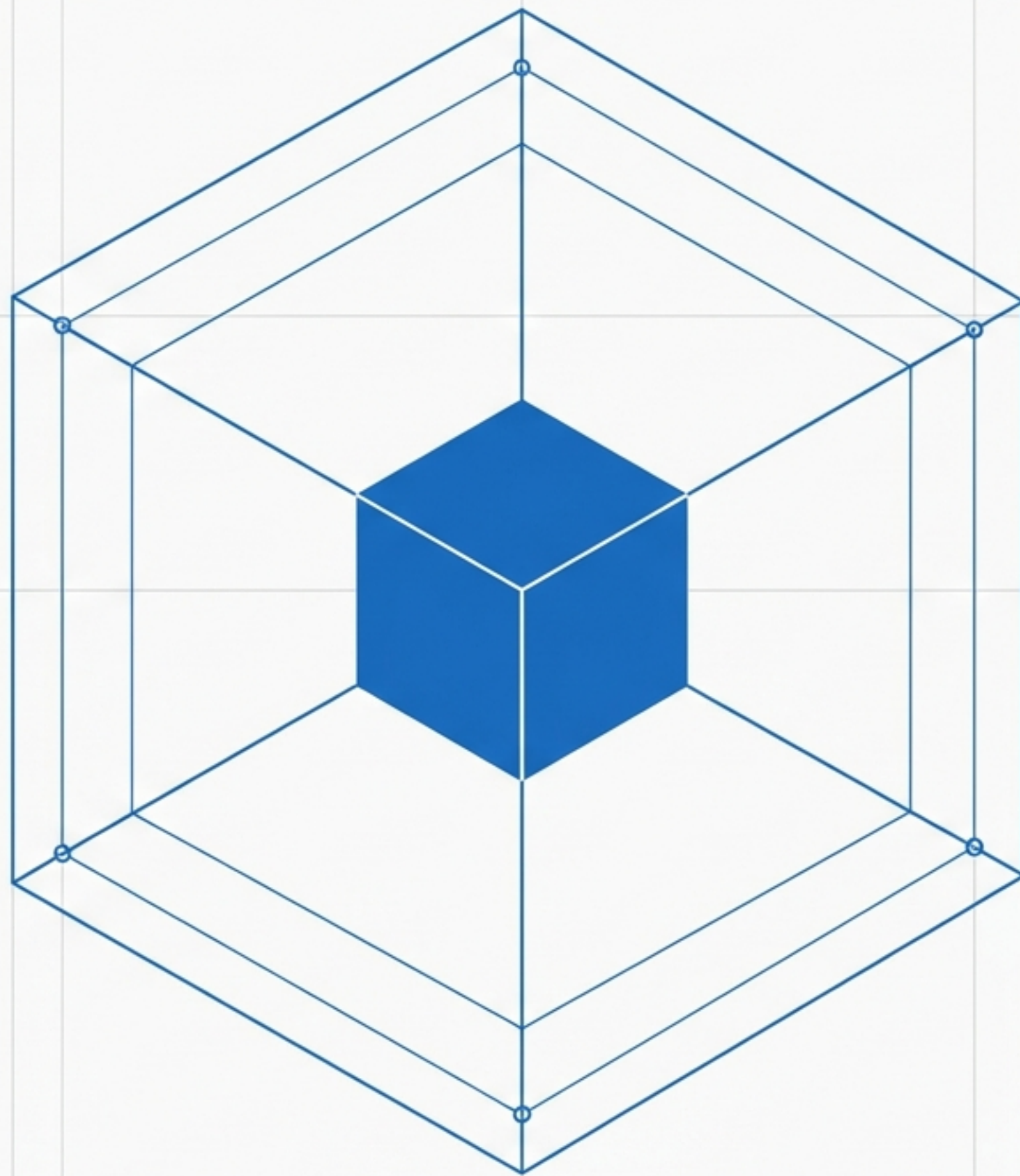
HTML Store & Retrieval System

A Unified Framework for Observable,
Versioned HTML Caching

VERSION: v1.4.53

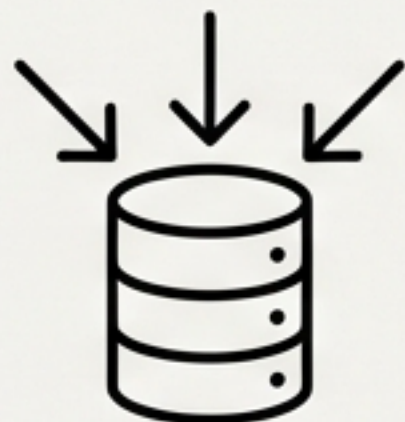
DATE: January 20, 2026

PACKAGE: mgraph_ai_service_html_graph



Beyond Simple Storage: The Need for an Intelligent Cache

Modern applications require more than just saving a page. The system bridges the gap between HTML capture and HTML consumption, providing a unified, observable approach that works seamlessly across REST APIs and internal services.



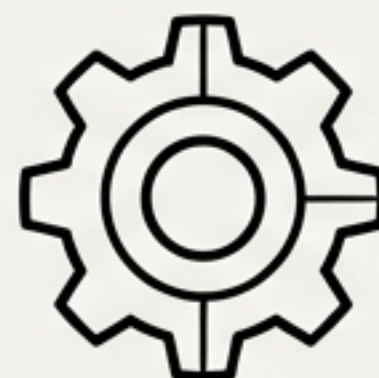
Multi-mode Storage

Store from raw content, URLs, or explicit keys.



Automatic Versioning

Track page evolution over time with unique IDs.



Transformation

Modify HTML (filtering, ad removal) before serving.



Observability

Full visibility into processing pipelines via FLeT.

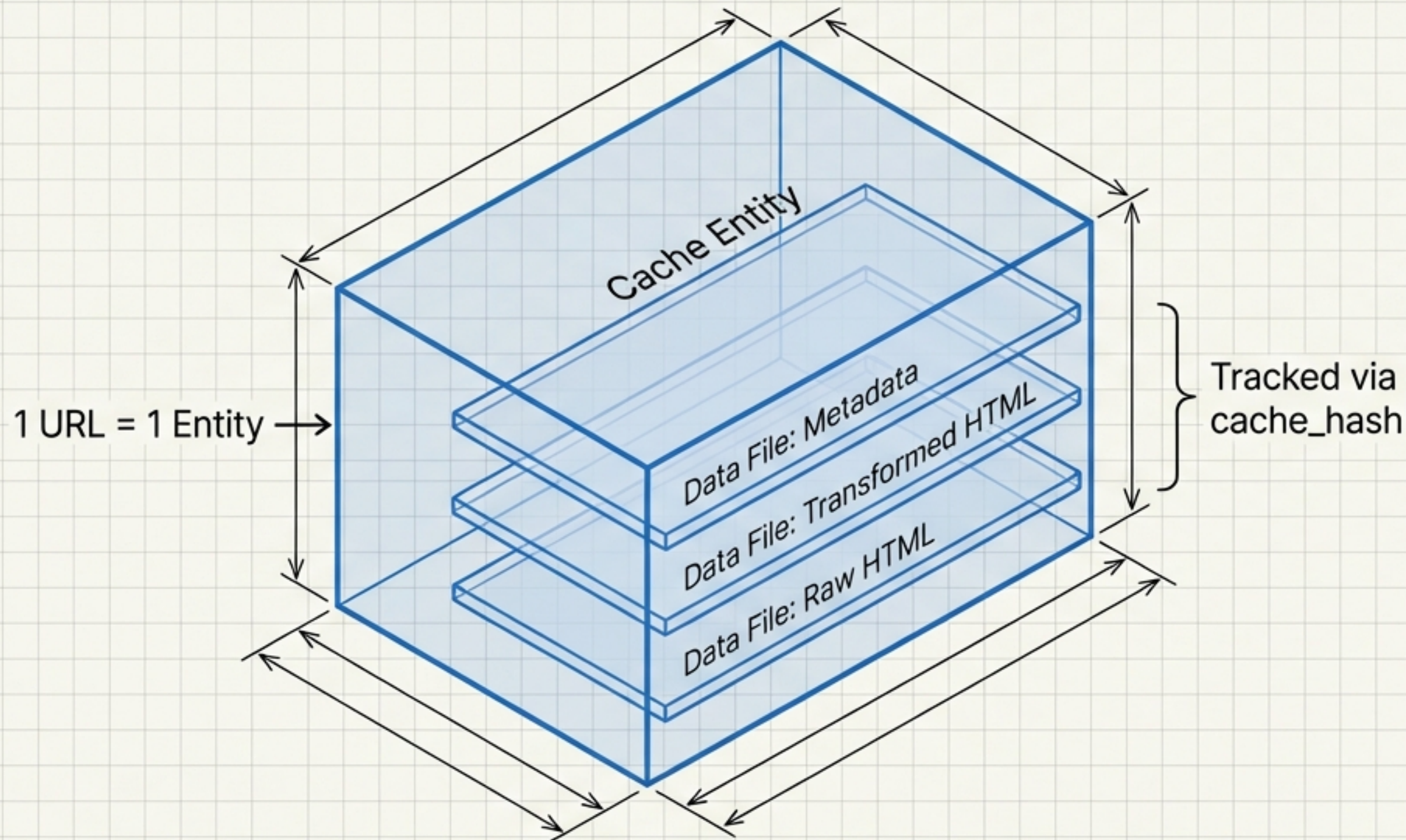


Decoupling

Separate the act of capturing from the act of consuming.

The Page Entity Model

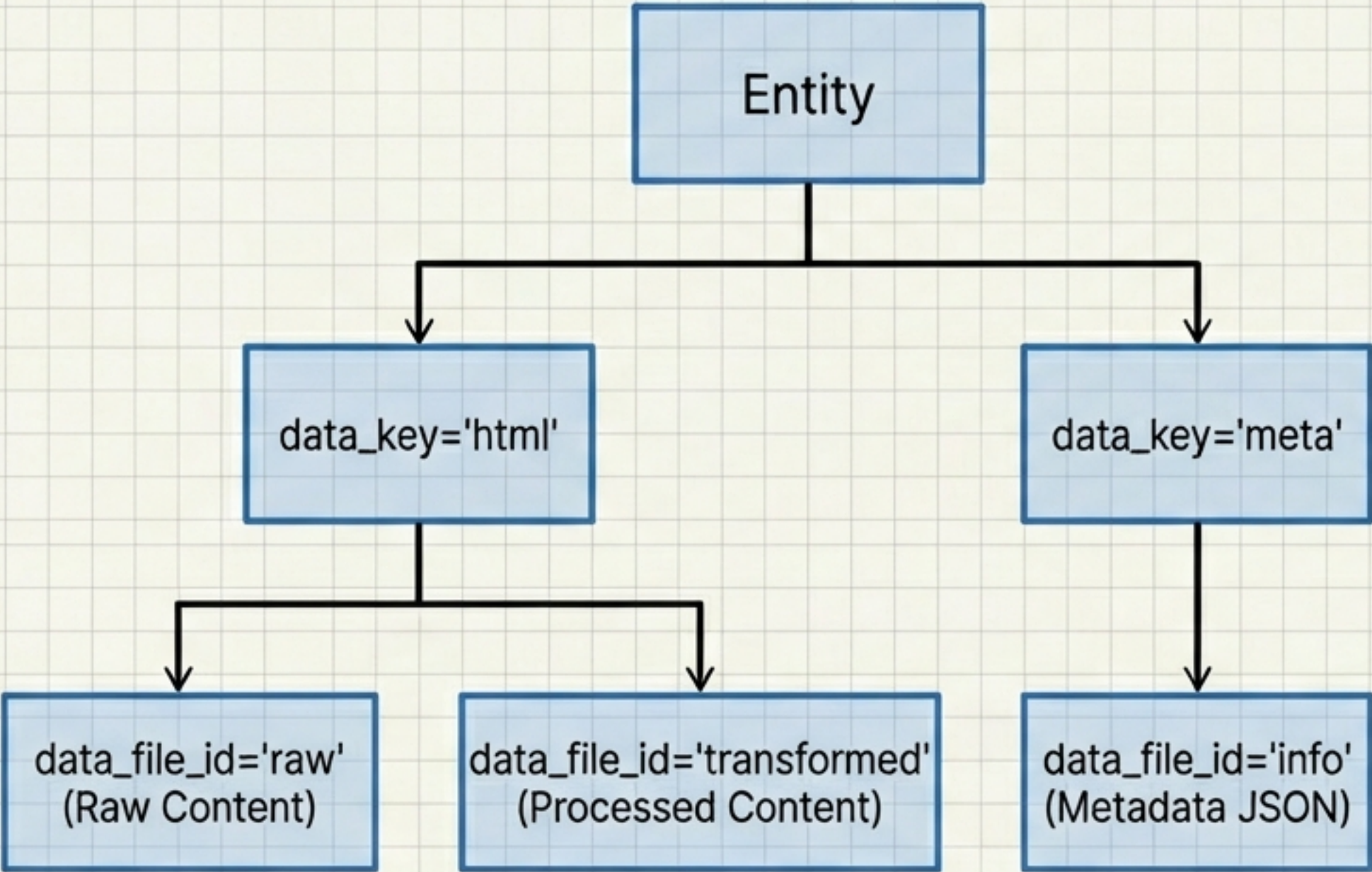
Moving from Key-Value Pairs to Structured Entities



- Every HTML page becomes an entity in the Cache Service.
- HTML content is stored as a data file within that entity.
- This separation allows a single entity to hold raw HTML, transformed HTML, and metadata simultaneously.

Internal Storage Logic & File Organisation

How files are structured within an Entity



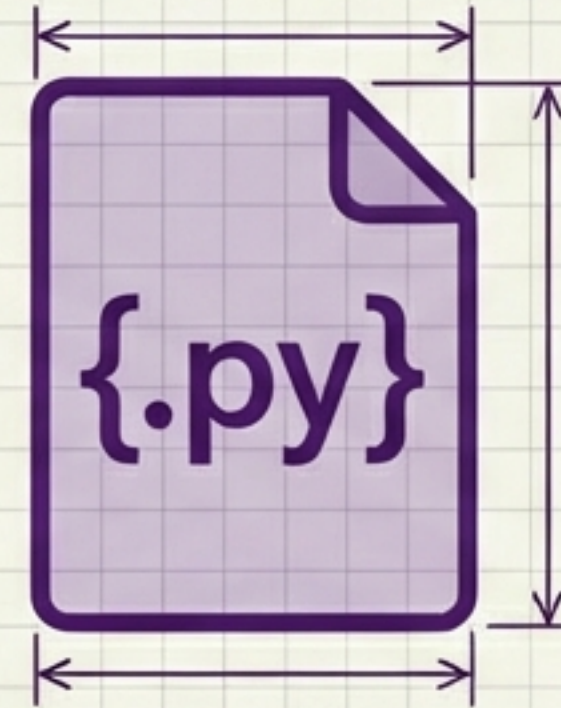
Content Type	Data Key	File ID
Raw Content	html	raw
Processed Content	html	transformed
Metadata	meta	info

Two Integration Pathways



Path A: External Integration

- **User:** External clients, loosely coupled systems.
- **Method:** REST API
- **Protocol:** HTTP / JSON



Path B: Internal Integration

- **User:** Services within mgraph_ai_service_html_graph
- **Method:** Direct Service Import
- **Protocol:** Python Object Calls

REST API: Storage Operations

Base URL: `/flet-html-domain/html/store/{ns}/...`

1. Store Raw HTML

POST `.../raw`

Auto-generates the key.
Best for pure content
injection.

2. Store from URL

POST `.../url`

Fetches content from
origin. Uses URL as key.

3. Store with Explicit Key

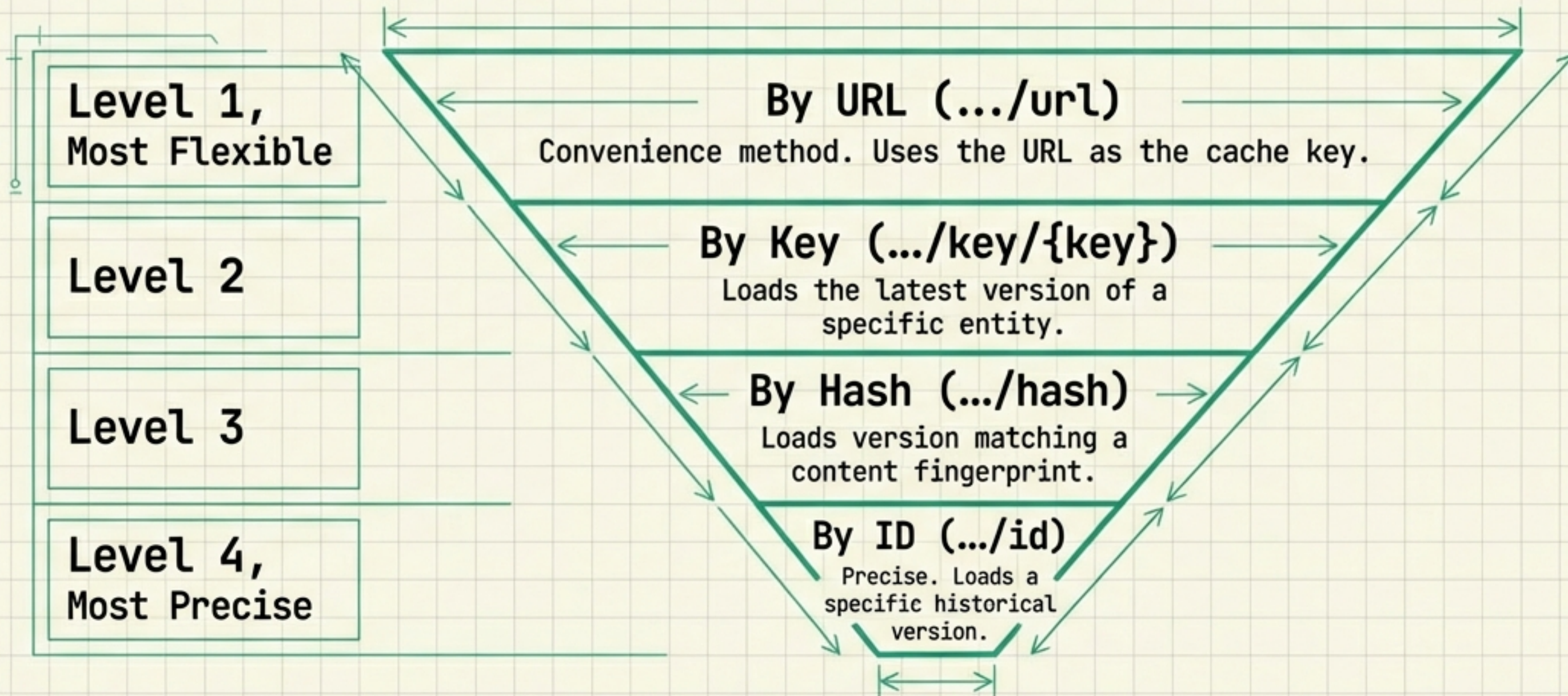
POST `.../key/{key}`

User defines the
specific identifier.

Success Response returns `'cache_id'` and `'cache_key'`.

REST API: Load Operations

Base URL: `/flet-html-domain/html/load/{ns}/...`



Namespace ({ns}) is required for all operations to ensure isolation.

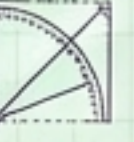


Internal Service Integration

Direct Python implementation for mgraph services

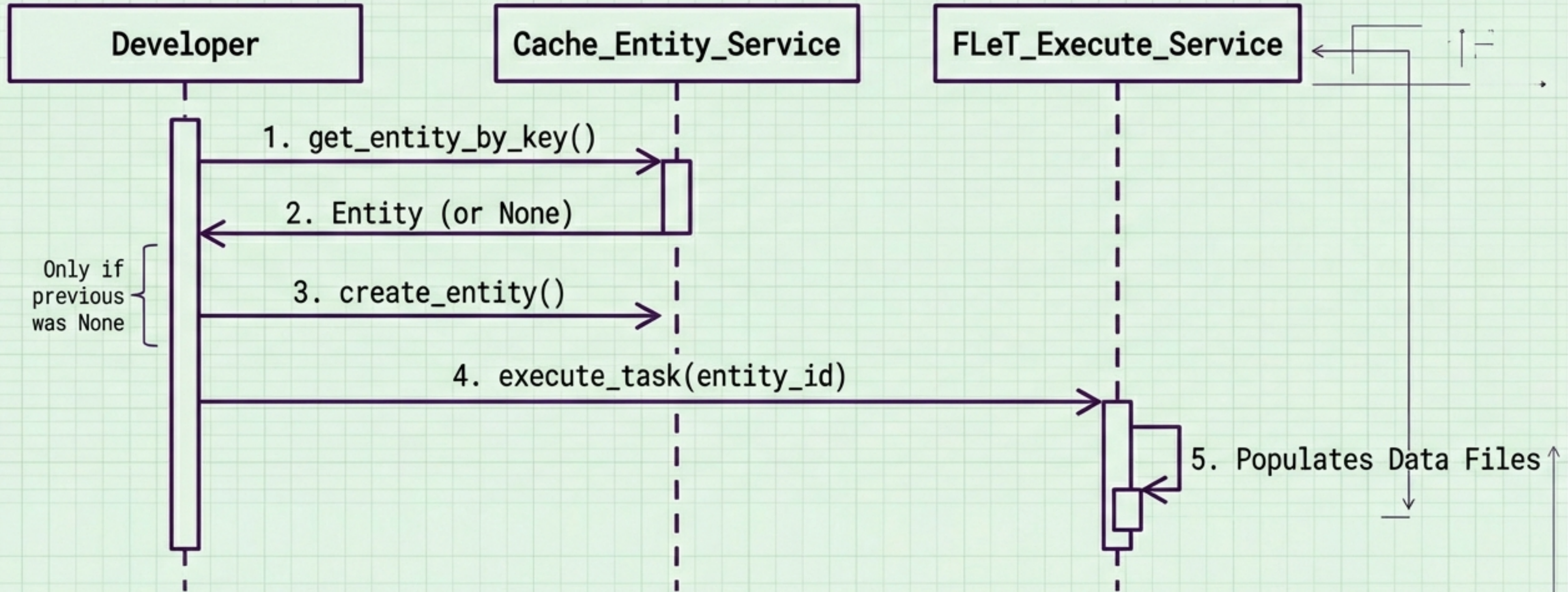
```
1 from mgraph_ai_service_html_graph.domain.FLeT__Html__Domain__Service import
2 import FLeT__Html__Domain__Service
3
4 # Initialize
5 domain_service = FLeT__Html__Domain__Service()
6
6 # Store Operation
7 domain_service.store_from_url(
8     url='https://example.com',
9     namespace='project-alpha'
10 )
11
12 # Load Operation
12 entity = domain_service.load_by_key(
13     key='https://example.com',
14     namespace='project-alpha'
15 )
```

Internal methods mirror the REST endpoints 1:1 but bypass HTTP overhead.

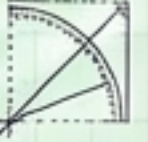


Advanced Pattern: Deferred Execution

For complex workflows, decouple entity creation from data processing.



Benefit: Allows granular control over `data_key` and `data_file_id` assignment.



MVP Use Case: MitmProxy Passive Mode

Zero-Config Caching & Archival

Intercept & Save to Cache (Background)



The Workflow:

1. First Request: MitmProxy fetches from Origin, saves to Cache, returns to User.
2. Subsequent Requests: Loads directly from Cache.

Value: Browsing the web automatically builds your dataset.

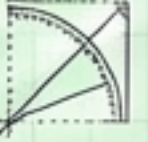


Proxy Configuration & Benefits

Setting	Default	Function
namespace	mitmproxy-cache	Isolates cache storage.
transform_mode	False	Enables content filtering.
include_query_params	False	Determines if ?q= affects key.

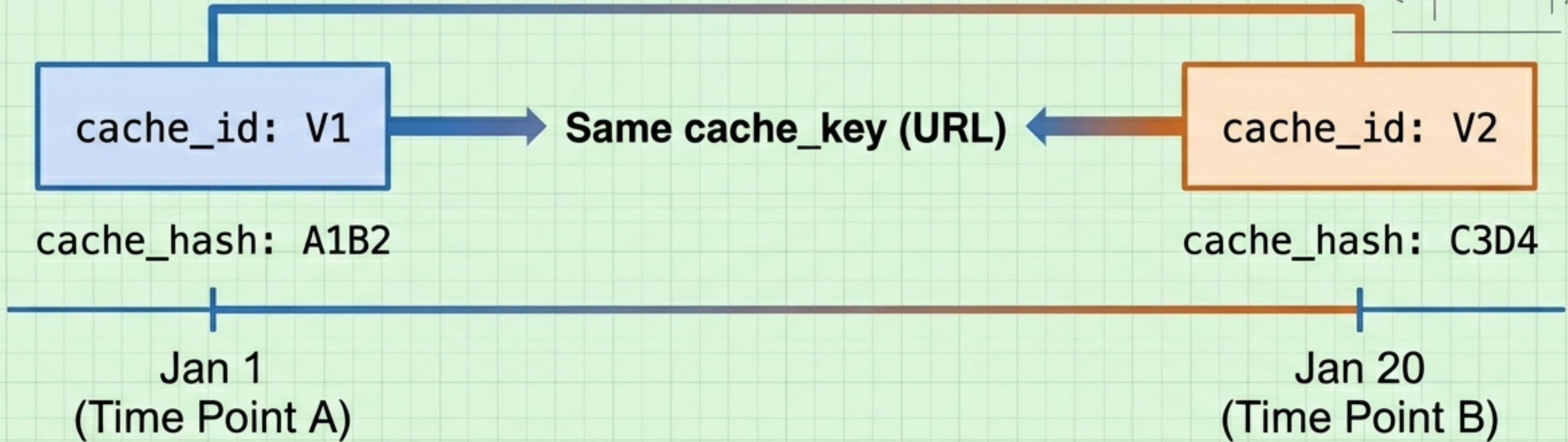
Why use Passive Mode?

- **Zero-config:** No manual API calls required.
- **Offline Access:** Access previously visited pages without internet.
- **Time Travel:** Re-visiting changed pages creates new versions.
- **Transformation Ready:** Pipeline ready for content injection.

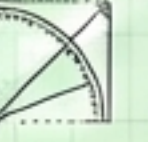


Versioning & History

Tracking the evolution of a page

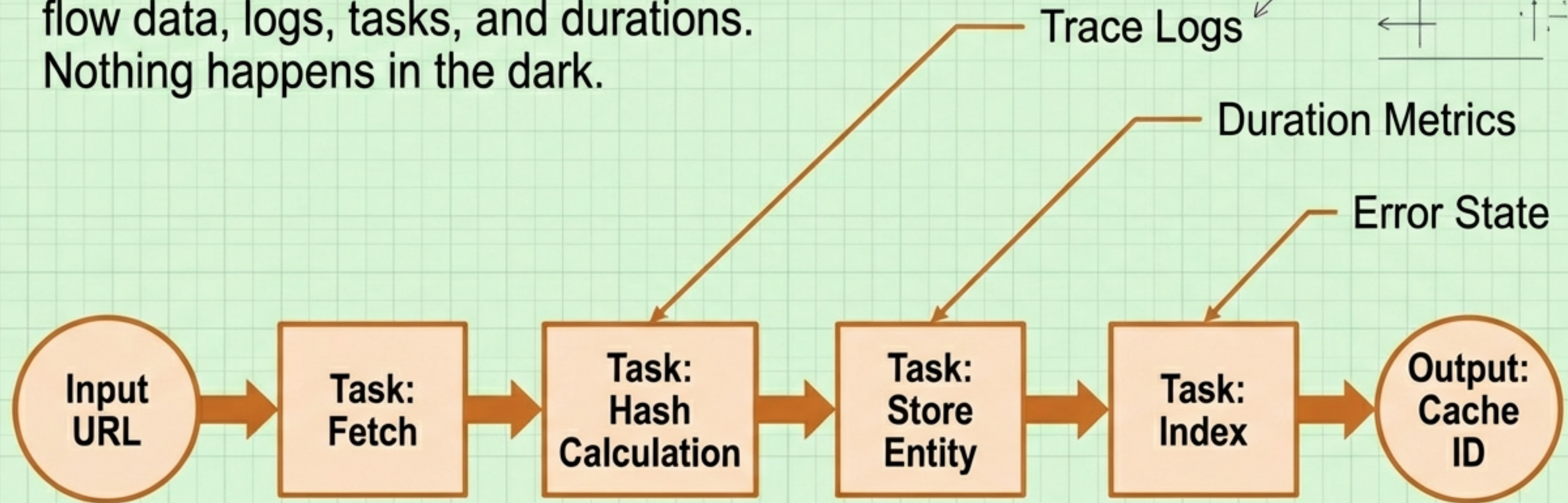


The system creates a **new cache_id** for **every unique** state of the page, allowing you to load historical versions by ID or the latest version by Key.



Observability with FLeT

Every Store and Load operation records flow data, logs, tasks, and durations. Nothing happens in the dark.

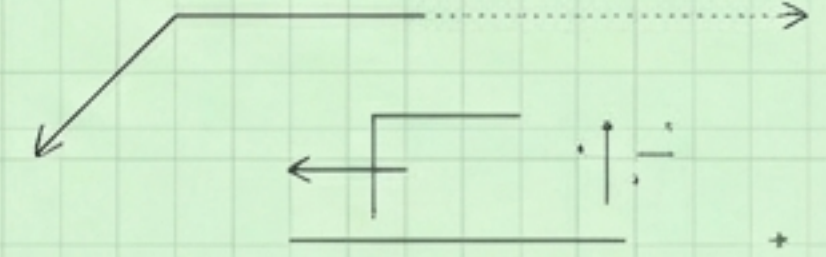
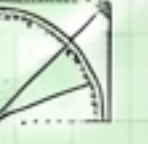


Inspect FLeT execution logs to debug pipelines or analyse performance bottlenecks.



The Developer's Cheat Sheet

OPERATION	REST ENDPOINT	INTERNAL FUNCTION
Store Raw	.../store/{ns}/raw	store_raw()
Store URL	.../store/{ns}/url	store_from_url()
Store Key	.../store/{ns}/key/{key}	store_with_key()
Load ID	.../load/{ns}/id	load_by_id()
Load Key	.../load/{ns}/key/{key}	load_by_key()
Load URL	.../load/{ns}/url	load_by_url()
Load Hash	.../load/{ns}/hash	load_by_hash()



Error Handling & Reliability

200 OK: Success.

400 Bad Request: Empty HTML or invalid URL.

404 Not Found: Entity does not exist (Handle cache miss).

500 Server Error: Internal failure (Retry).

The HTML Store & Retrieval System provides the structural foundation for robust, observable, and versioned web data consumption.